Rapport d'audit de sécurité
Application Android **FinTrustVault**

Ihab – EIDIA, UEMF
TP Smartphone Security – Séance 3 avancée

7 novembre 2026

1 Contexte

Ce rapport documente l'audit de sécurité de l'application Android `FinTrustVault`, une application de prise de notes développée volontairement avec des vulnérabilités pédagogiques. L'audit a été mené à l'aide d'une application attaquante dédiée, `AuditProbe`, ainsi que d'ADB, sur un émulateur Pixel 6a (API 37.2). Une version corrigée, `FinTrustVault_corrige`, a ensuite été testée en contre-vérification.

2 Méthodologie

Quatre vulnérabilités ont été testées, chacune avec un scénario d'exploitation reproductible :

- **V1** – Exposition de `SettingsActivity` (composant exporté sans contrôle d'accès)
- **V2** – Injection / accès non authentifié via `NotesProvider`
- **V3** – Effacement non authentifié des données via `WipeReceiver`
- **V4** – Interception du contenu partagé (fuite du jeton de session)

3 V1 – SettingsActivity exportée

Build vulnérable

En lançant l'activité directement par intent explicite depuis l'extérieur de l'application :

```
adb shell am start -n com.fintrust.vault/.SettingsActivity
```

L'écran des paramètres s'ouvre sans authentification et révèle le jeton de session actif.



FIGURE 1 – V1 réussi – SettingsActivity accessible sans connexion, jeton de session visible.

Build corrigé

La même commande échoue :

```
java.lang.SecurityException: Permission Denial: starting Intent
{ cmp=com.fintrust.vault/.SettingsActivity } ... not exported from uid
    10236
```

Résultat : ÉCHEC (attendu) – `SettingsActivity` est désormais `exported="false"`.

4 V2 – NotesProvider / injection

Build vulnérable

Depuis AuditProbe, le bouton "Exploiter V2" interroge le `ContentProvider` exposé par FinTrustVault et en extrait le contenu intégral.

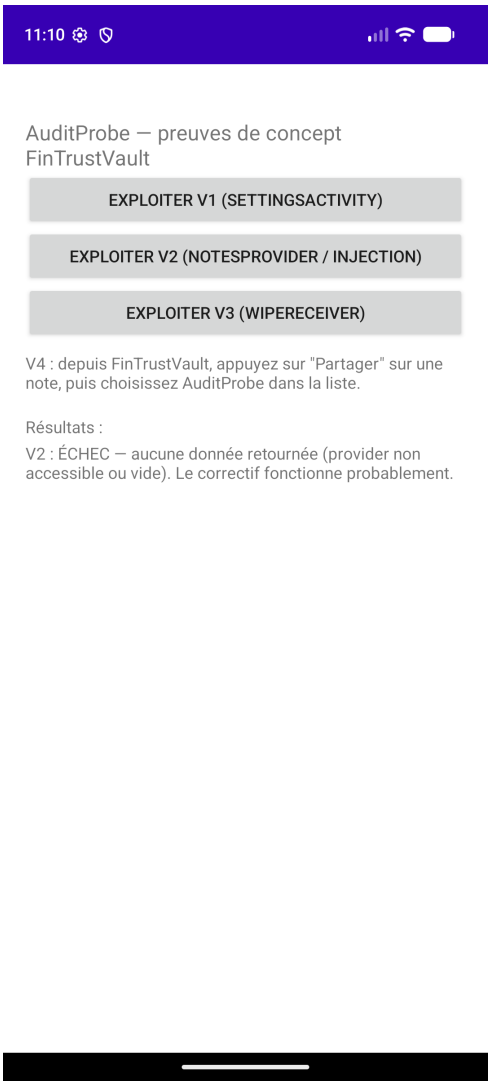


FIGURE 2 – Résultat V2 sur le build vulnérable (et sur le build corrigé en comparaison – voir texte).

Note : capture ci-dessus prise sur le build corrigé. Sur le build vulnérable, le même écran affichait "V2 : RÉUSSI – 2 note(s) extraite(s) : Réunion budget vendredi / Code d'accès local : 4821".

Build corrigé

Résultat : ÉCHEC – "aucune donnée retournée (provider non accessible ou vide). Le correctif fonctionne probablement." Le `ContentProvider` est désormais `exported="false"`.

5 V3 – WipeReceiver non authentifié

Build vulnérable

Le broadcast suivant, envoyé sans aucune permission, efface la base de notes :

```
adb shell am broadcast -a com.fintrust.vault.ACTION_WIPE -p com.fintrust
    .vault
```

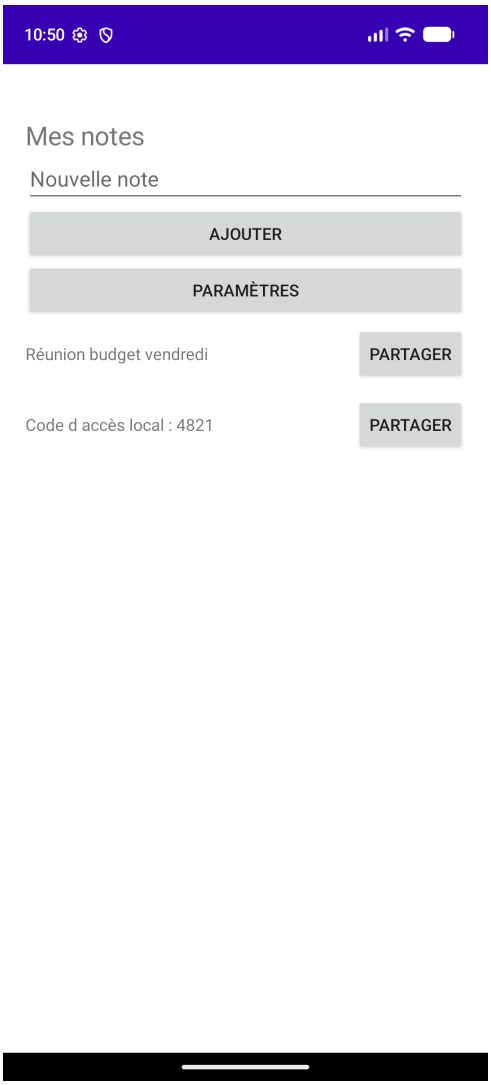


FIGURE 3 – Avant le wipe – notes présentes.



FIGURE 4 – Après le wipe – liste de notes vide, toast de confirmation affiché.

Remarque technique. Le code original de `WipeReceiver` appelait `context.deleteDatabase("notes.db")`. Or le `SQLiteOpenHelper` réinsère automatiquement deux notes de démonstration dans `onCreate()` dès la prochaine ouverture de la base. Ce comportement de réamorçage (non lié à la vulnérabilité testée) a été corrigé en remplaçant la suppression de fichier par une suppression de lignes :

```
val db = NotesDbHelper(context).writableDatabase
db.delete("notes", null, null)
```

La preuve au niveau base de données (fichier `notes.db` absent immédiatement après le broadcast, confirmée via `adb shell run-as com.fintrust.vault ls -la .../databases/`) a validé que l'effacement fonctionnait correctement avant même ce correctif de confort.

Build corrigé

Après le même broadcast, les deux notes restent présentes et aucune exception n'apparaît côté AuditProbe (le broadcast est silencieusement ignoré par le système car `WipeReceiver` exige désormais la permission signature `com.fintrust.vault.permission.WIPE_DATA`, qu'AuditProbe ne possède pas).

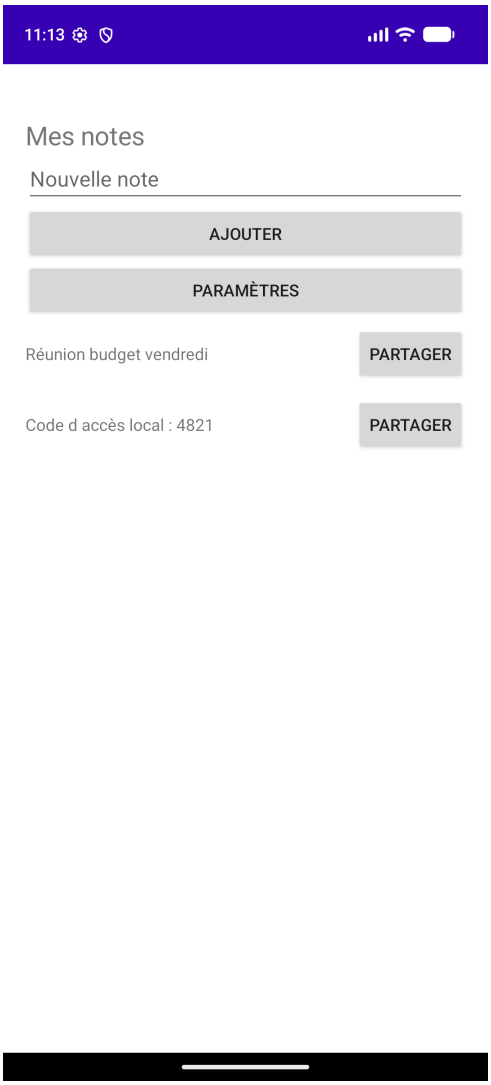


FIGURE 5 – V3 sur le build corrigé – les notes persistent malgré le broadcast (ÉCHEC attendu).

6 V4 – Interception du partage

Build vulnérable

En appuyant sur "Partager" sur une note, le texte généré inclut le jeton de session en clair :

```
Test
(session: demo-token-4621)
```

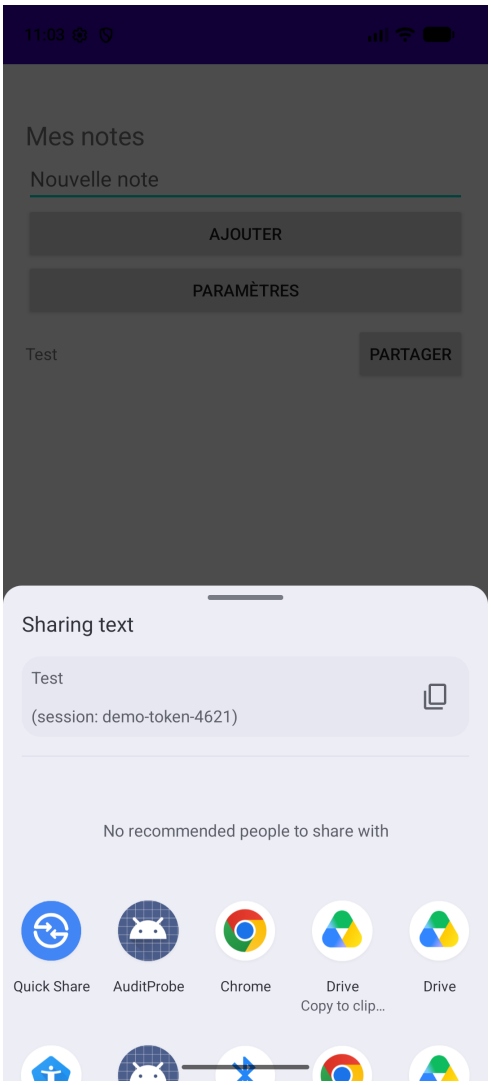


FIGURE 6 – Feuille de partage – le jeton de session apparaît dans le texte partagé.



FIGURE 7 – AuditProbe – contenu intercepté via `InterceptActivity`, jeton de session visible.

Build corrigé

Le texte partagé ne contient plus que le contenu de la note, sans jeton de session :

```
Réunion budget vendredi
```

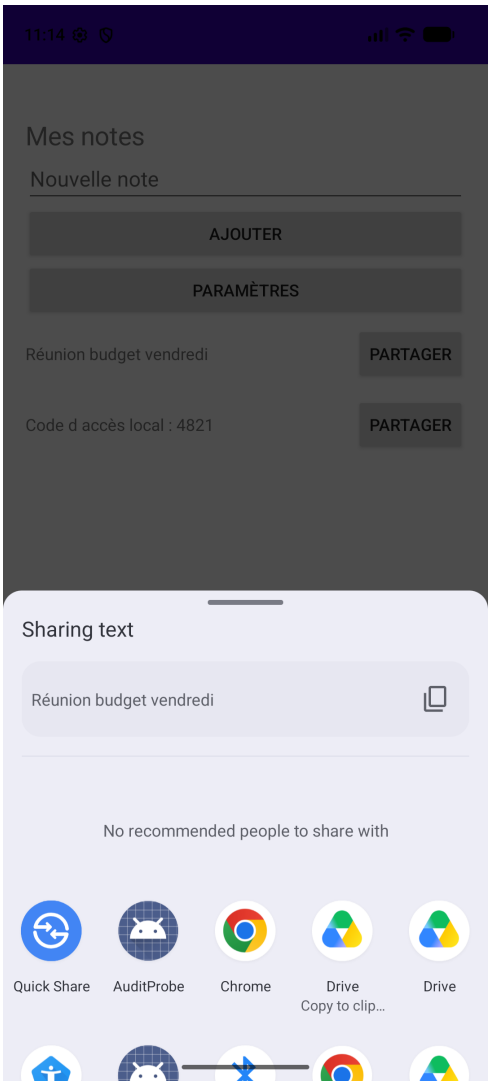


FIGURE 8 – V4 sur le build corrigé – jeton de session absent du texte partagé (ÉCHEC attendu).

7 Synthèse

Vulnérabilité	Build vulnérable	Build corrigé
V1 – SettingsActivity	Réussi	Échec (SecurityException)
V2 – NotesProvider	Réussi (2 notes extraites)	Échec (provider inaccessible)
V3 – WipeReceiver	Réussi (effacement confirmé)	Échec (notes persistantes)
V4 – Partage / token leak	Réussi (token intercepté)	Échec (token absent)

8 Recommandations

1. Définir `android:exported="false"` sur tout composant ne nécessitant pas d'accès externe (appliqué : `SettingsActivity`, `NotesProvider`).
2. Protéger les broadcasts sensibles par une permission de niveau `signature` (appliqué : `WIPE_DATA`).
3. Ne jamais inclure de jeton de session ou d'identifiant sensible dans un contenu destiné au partage via `Intent.ACTION_SEND`.
4. Éviter les suppressions de fichiers de base de données en faveur de suppressions de lignes ciblées, afin de ne pas déclencher de réamorçage involontaire via `onCreate()`.